

LC 410: Split Array Largest Sum

© Authored by Arpit Raj, LNMIIT Jaipur | August 26, 2026

PROBLEM OVERVIEW

Companies: De Shaw, Tekion, Pornhub

Need: to split array into k subarrays such that largest sum of any subarray is minimized.

O/P: minimized largest sum of split

EXAMPLE & INTUITION

Array: `[7, 2, 5, 10, 8]`, $k = 2$

`[7 | 2 5 10 8]` → 25

`[7 2 | 5 10 8]` → 23

`[7 2 5 | 10 8]` → **18** → minimum → ans

For $k = 2$ it was easy to visualize and $O(n^2)$ total, but what if $k = 50$?

DP APPROACH

Let DP state be $dp[i][k]$ which means **minimum possible largest subarray sum** when first i elements are split into k parts.

now let last part start-at j index, then:

$[0 \dots j-1] \mid [j \dots i-1]$

now:

- left = $dp[j][k-1]$
- right = $pref[i] - pref[j]$

largest sum for this split $\rightarrow \max(left, right)$

$$dp[i][k] = \min(dp[i][k], \max(dp[j][k-1], pref[i]-pref[j]))$$

Base Case:

$$dp[0][0] = 0$$

LOOP STRUCTURE & COMPLEXITY

```
for (int i = 1; i <= n; i++) {
    for (int parts = 1; parts <= k; parts++) {
        // need to try every j
        for (int j = parts - 1; j < i; j++) {
            ...
        }
    }
}
```

Hence:

- TC = $O(n^2k)$
- SC = $O(nk)$ → DP

C++ IMPLEMENTATION

```
class Solution {
public:
    int splitArray(vector<int>& nums, int k) {
        int n=nums.size();

        //classic partition dp problem
        //dp[i][k] first i elements in k parts
        //and then change the index j of last part
        //[0 ..... j-1] [j ..... i-1]

        //max sum ya to dp[j][k-1] ka hoga
        //ya to pref[i]-pref[j] ka hoga
        //dp[i][k]=min(dp[i][k],max(cand1,cand2))

        vector<int> pref(n+1,0);
        for(int i=0;i<n;i++){
            pref[i+1]=pref[i]+nums[i];
        }

        vector<vector<int>> dp(n+1,vector<int>(k+1,INT_MAX));
        dp[0][0]=0;

        for(int i=1;i<=n;i++){
```

```

        for(int parts=1;parts<=k;parts++){

            for(int j=parts-1;j<i;j++){
                //j start from parts-1 because first j elements need to
                be div to parts-1 non empty parts
                //and j<i cause in last part [j .... i-1] and [i ....
                i-1] no sense

                int cand1=pref[i]-pref[j];
                int cand2=dp[j][parts-1];

                int voted=max(cand1,cand2);
                dp[i][parts]=min(dp[i][parts],voted);
            }
        }

        return dp[n][k];

    }
};

```